

DEMO BLOG

Teaching a Spider to Find a Pink Box

What an artist learned by building a reinforcement-learning experiment without pretending to be an engineer

By Sanno - demonstration draft

I began with a creature and a question: when an enemy in a game looks intelligent, what is actually happening underneath?

I did not begin this project as a programmer

I come from an art and game-design direction. I understand shape, movement, silhouette, mood, and whether a creature feels threatening or ridiculous. I did not understand reinforcement learning, Python, PPO, policies, rollouts, entropy, or why a robot would decide that standing completely still was a successful strategy.

That last part became important later. At the beginning, I was simply interested in a Leaper-like enemy: a mechanical creature that could chase a player, hide, shoot, recover from damage, and perhaps behave in ways I had not explicitly designed. I wanted to know how the little creature thought.

My first lesson was that it probably did not think in the way I imagined. Most game enemies are not trained. They use behavior trees or state machines: carefully written rules such as, if the player is visible, chase; if health is low, hide; if close enough, attack. That approach is predictable and useful. Reinforcement learning is different. Instead of writing the moves, you create a world, define what the creature can sense and do, and give it feedback. It experiments until it develops a strategy.

A behavior tree is closer to directing an animation. Reinforcement learning is closer to setting up a strange rehearsal room, establishing consequences, and watching the performer invent a routine. The second option was more complicated and less reliable. It was also the point of the project: I wanted to learn how learned behavior was made.

Turning technical words into design questions

The basic vocabulary became manageable when I stopped treating it as computer science and translated it into design. The agent was the character. The environment was the stage. Observations were its senses. Actions were the movements its body was allowed to perform. Reward was the direction I gave through consequences. The policy was the behavior it eventually developed.

This was the first moment when the work started to feel connected to art. Choosing what the Leaper could observe was like deciding whether a character had tunnel vision, memory, or awareness.

Choosing its actions was designing its body. Choosing rewards was shaping its personality. An aggressive creature and a cautious creature might have the same model and animations but value different outcomes.

I originally imagined far too much: shooting, hiding, injured legs, damaged eyes, water hazards, recovery, and different body parts with different consequences. These were useful design ideas, but they were not a useful first experiment. The first real Leaper needed one job: reach a stationary pink box.

The proof of concept was a blackout

In 3D work, I would not begin a scene with eyelashes and surface scratches. I would begin with a blackout. The same rule saved this project. We built a browser proof of concept using Three.js: a bounded world, a procedural six-legged robot, obstacles, cameras, controls, and a bright pink cuboid as the target.

The earliest version was a large single HTML file. It worked, which was enough at first. Later, it was divided into smaller pieces: one area for the robot, another for the world, and others for cameras, controls, simulation, and the training viewer. Git became a second kind of safety net. Each meaningful state could be saved, described, and recovered instead of being overwritten by the next experiment.

Visually, the result looked much more advanced than the learning problem underneath. This caused one of our most important misunderstandings. On screen, Leaper was a 3D hexapod walking across a field. It was easy to look at that image and say that we were training a six-legged robot to walk. We were not.

Python was training a simplified navigation policy. The learned controls were forward or reverse throttle and turning on a flat plane. The browser reconstructed a walking gait over the path, making the navigation disc look like a walking creature. The AI was not controlling individual joints, learning balance, or recovering from a fall. The body was visually three-dimensional; the learned decision problem was mostly two-dimensional.

That distinction is not an embarrassment. It is what made the experiment achievable. But it is important to describe honestly, because a convincing visualization can easily make a prototype appear to solve more than it actually solves.

Two worlds and one nervous brain

The visible world lived in JavaScript. The learning world lived in Python because the reinforcement-learning tools there were more mature: Gymnasium for defining the environment, Stable-Baselines3 for PPO, and TensorBoard for graphs. I initially thought the JavaScript environment and model could simply be exported into Python. Instead, Python needed its own simplified reconstruction of the relevant rules.

That created two worlds. If movement speed, timing, obstacles, collisions, or observations differed, the policy could learn perfectly in Python and behave terribly when used elsewhere. This is

sometimes called a simulation gap. In plain language, the creature studied for one exam and was handed another.

We began documenting the contract between those worlds: what Leaper observes, what each action means, how far it can move, how fast it can turn, what counts as a collision, and what ends an episode. The project stopped being only a visual prototype and became an experiment that needed its own records.

How to reward a creature without telling it what to do

My first instinct was to reward the Leaper only when it touched the pink target. The problem was that a distant reward did not tell a randomly moving agent which of its thousands of bad actions were slightly less bad. It could wander for a long time without ever experiencing success.

We added a progress signal based on the change in distance. If Leaper became closer to the target during a step, it received a small positive value. If it became farther away, it received a small negative value. The formula eventually used was $0.2 \times \text{previous distance} - \text{current distance}$. Reaching the target gave +25. Every step cost -0.01, and a collision cost -0.18.

That led me to a question that turned out to be better than simply asking for a bigger reward. What if the correct route required temporarily moving away from the target to get around an obstacle? A straight ruler through a wall says the target is close, even though the walkable route may be long. I learned the terms Euclidean distance, for the straight line, and geodesic distance, for the path around obstacles. I also learned about flood-fill, which can spread distances across a grid like paint spreading around walls.

We did not ultimately implement geodesic shaping because the current arena had only a few sparse obstacles. The discussion still mattered. It taught me that rewards are not neutral measurements. They contain assumptions about what progress means.

The graph that looked like success

Training produced too much information to understand by watching the creature alone. We printed episode rewards, recorded diagnostic files, used TensorBoard, and eventually built a dedicated browser training view. It could replay checkpoints and actual PPO experience, select workers and episodes, scrub through frames, and import earlier runs.

This created another important distinction: an attractive replay is an example, not a measurement. During training, PPO intentionally explores. It sometimes produces exciting behavior that its final deterministic policy will not repeat. We therefore separated stochastic training results from deterministic evaluation, where the policy is tested consistently without exploratory randomness.

The strongest early run was PPO_2. With only body collision, it reached 60 percent deterministic success at the 300,000-step checkpoint. Then we made collision more realistic by checking the body plus points along all six legs. Performance collapsed. PPO_3 and PPO_4 showed that the task had become much harder. This was frustrating, but useful: a success in the simpler world had not disappeared mysteriously. We had changed the problem.

Why Leaper learned to do nothing

PPO_8 used the cleaner distance-change reward, the time cost, the collision cost, and the +25 goal reward. After more than 300,000 collected steps, deterministic success was still zero. The Leaper barely moved.

The phrase zero throttle sounded like a broken control. In reality, it meant the policy was choosing a forward command very close to zero. The agent had learned that movement was dangerous. Moving could create repeated collision penalties. Standing still cost only the small time penalty. From the agent's perspective, doing nothing was not laziness. It was the safest available strategy.

At first, it was tempting to increase the target reward or let training run longer. Diagnostics showed why that would have been guesswork. The target was physically reachable: typical spawns needed only about 39 to 43 full-speed steps, while an episode allowed 400. The +25 reward already dominated any individual penalty. Collision did not end the episode. The actual behavior was a mixture of standing still and, in a few episodes, repeating the same rejected collision until timeout.

The most useful question became: under the rules we gave it, why does this bad behavior make sense? That question changed the project. We stopped treating a failed policy like a disobedient creature and started treating it as evidence about the environment.

Changing one thing at a time

PPO_9 added memory of the previous collision and the previous movement and turning actions. Everything else stayed the same. The exploratory training behavior became more active, but the deterministic policy still selected zero forward throttle. Collision memory alone was not the solution.

PPO_10 tested another explanation. Until then, forward throttle existed only between zero and one. The new action ranged from -1 to 1: negative meant reverse, zero meant stop, and positive meant forward. The policy started fresh because changing the meaning of an action makes older trained models incompatible.

This small change removed the literal freeze. During deterministic evaluation, stopped steps fell from almost 100 percent to zero. Leaper used both forward and reverse, made an average of 6.29 units of progress toward the target, and succeeded in 5 percent of episodes. In the latest stochastic training episodes it reached 34 percent success, but that number cannot be presented as the deployment result. The reliable deterministic result was still only 5 percent.

The fix also revealed the next problem. Deterministic collision steps rose to almost 39 percent, and some collision streaks lasted almost the entire 400-step episode. Leaper had regained movement but had not learned reliable recovery from contact. Solving the freeze did not finish the task. It exposed the obstacle underneath it.

What I actually learned

When I started, I thought the difficult part would be understanding an algorithm. The algorithm was mostly the sealed engine. The difficult part was designing a fair world, measuring the behavior honestly, and changing one meaningful thing at a time.

I also learned that my background was not separate from the technical work. An artist notices rhythm, hesitation, weight, repetition, and when movement looks wrong. A game designer asks whether an action feels intelligent, fair, readable, or exploitable. Those observations became diagnostic questions. The numbers helped confirm them, but the questions often began with watching.

The project is not finished. It is also not the failed experiment that zero percent success once appeared to describe. It now has a modular simulator, a Python training pipeline, controlled evaluations, detailed collision and reward diagnostics, replay tools, saved checkpoints, and a written history of PPO_1 through PPO_10. The next responsible step is not to declare victory or train blindly for another 300,000 steps. It is to repeat PPO_10 with two new random seeds and see whether signed throttle produces the same improvement consistently.

If it does not, the next experiment will examine collision recovery: allowing Leaper to turn or reverse away from contact instead of rejecting movement and rotation together. One change, one comparison, one new piece of evidence.

I began by asking how a game creature thinks. The more useful answer was that it does not understand the target, the rocks, or my intentions. It understands the opportunities and consequences I construct around it. Learning to see those rules clearly - and to admit when my explanation was wrong - became the real project.

Author's note

This demonstration draft intentionally keeps the voice personal and the technical claims conservative. It treats mistakes and corrections as part of the story rather than hiding them. Exact experiment figures should continue to be checked against TRAINING.md before publication.